AWR Analysis Report

NEODBPRD / neodbprd_1 · 13 snapshots · Generated 2026-08-21T11:57:34Z

DB NAME	INSTANCE	RELEASE	HOSTNAME	PLATFORM	CPUS / MEMORY	RAC / CDB
NEODBPRD	neodbprd_1	19.0.0.0.0	TCLFSLPRDDB1	Linux x86 64-bit	76 / 589.62 GB	YES / —

90.7 / 100

GREEN

Health Score

Overall Database Health

HOST CPU	BUFFER CACHE	WAIT EVENTS	PARSE PRESSURE
18.4/25	24.9/25	23.8/25	23.6/25

Scoring: CPU load · Buffer hit% · Wait dominance · Hard parse rate
GREEN ≥75 · AMBER 50–74 · RED <50

WORKLOAD PROFILE — AVERAGES ACROSS ALL SNAPSHOTS

PERFORMANCE KPIS

ALL TIME METRICS IN SECONDS/SECOND (S/S) · HOST CPU FROM `AWR_OS_STATISTICS` (BUSY_TIME / TOTAL CPU TIME)

AVG DB TIME	AVG DB CPU	AVG TPS	AVG LOGICAL READS	AVG PHYSICAL READS	AVG HARD PARSES	AVG HOST CPU
21.02	18.61	116.4	2529264	46236	2.9	26.4%
seconds / second (s/s)	seconds / second (s/s)	transactions / second	blocks / second	blocks / second	per second	busy% · I/O wait 1.2%

DB Time & DB CPU: from `awr_load_profile` (Oracle Load Profile per_sec column) · Host CPU: derived from `awr_os_statistics` BUSY_TIME ÷ (BUSY_TIME + IDLE_TIME) per snapshot · s/s = aggregate seconds of that resource consumed per 1 second of elapsed time (across all sessions)

AWR SNAPSHOT SUMMARY

Snap	116883	116884	116885	116886	116887
	2024-02-21 09:00:49	2024-02-21 09:30:00	2024-02-21 10:00:11	2024-02-21 10:30:25	2024-02-21 11:00:40
DB TIME (S/S)	11.2	11.5	16.3	21.3	20.9
DB CPU (S/S)	10.5	10.8	14.7	18.8	18.8
TRANSACTIONS/S	38.7	57.2	91.8	117.6	123.9
LOGICAL READS/S	2170656	1466921	1950617	2524588	2540371
PHYSICAL READS/S	30642	30053	50212	62759	45962
HARD PARSES/S	0.8	1.4	2.4	3.3	3.2
HOST CPU BUSY%	14.9%	15.4%	20.9%	26.6%	26.7%

Snap 116888
2024-02-21 11:30:55

DB TIME (S/S)	DB CPU (S/S)
23.0	20.8
TRANSACTIONS/S	LOGICAL READS/S
140.7	2765984
PHYSICAL READS/S	HARD PARSES/S
53309	3.6
HOST CPU BUSY%	
29.5%	

Snap 116889
2024-02-21 12:00:10

DB TIME (S/S)	DB CPU (S/S)
24.4	21.9
TRANSACTIONS/S	LOGICAL READS/S
133.4	2895468
PHYSICAL READS/S	HARD PARSES/S
42865	4.0
HOST CPU BUSY%	
31.1%	

Snap 116890
2024-02-21 12:30:25

DB TIME (S/S)	DB CPU (S/S)
24.3	21.5
TRANSACTIONS/S	LOGICAL READS/S
136.8	2820181
PHYSICAL READS/S	HARD PARSES/S
45923	3.8
HOST CPU BUSY%	
30.6%	

Snap 116891
2024-02-21 13:00:41

DB TIME (S/S)	DB CPU (S/S)
23.4	21.0
TRANSACTIONS/S	LOGICAL READS/S
131.5	2777451
PHYSICAL READS/S	HARD PARSES/S
40693	3.2
HOST CPU BUSY%	
29.7%	

Snap 116892
2024-02-21 13:30:56

DB TIME (S/S)	DB CPU (S/S)
21.4	19.8
TRANSACTIONS/S	LOGICAL READS/S
123.4	2712435
PHYSICAL READS/S	HARD PARSES/S
32044	3.2
HOST CPU BUSY%	
28.1%	

Snap 116893
2024-02-21 14:00:09

DB TIME (S/S)	DB CPU (S/S)
24.7	20.8
TRANSACTIONS/S	LOGICAL READS/S
116.8	2683601
PHYSICAL READS/S	HARD PARSES/S
65358	2.6
HOST CPU BUSY%	
29.8%	

Snap 116894
2024-02-21 14:30:24

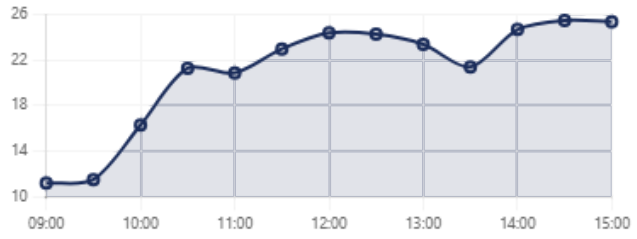
DB TIME (S/S)	DB CPU (S/S)
25.5	21.1
TRANSACTIONS/S	LOGICAL READS/S
149.5	2762378
PHYSICAL READS/S	HARD PARSES/S
56471	2.8
HOST CPU BUSY%	
30.0%	

Snap 116895
2024-02-21 15:00:39

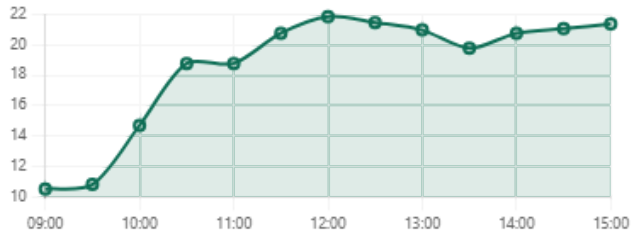
DB TIME (S/S)	DB CPU (S/S)
25.4	21.4
TRANSACTIONS/S	LOGICAL READS/S
151.7	2809780
PHYSICAL READS/S	HARD PARSES/S
44773	3.0
HOST CPU BUSY%	
30.5%	

TREND ACROSS SNAPSHOTS

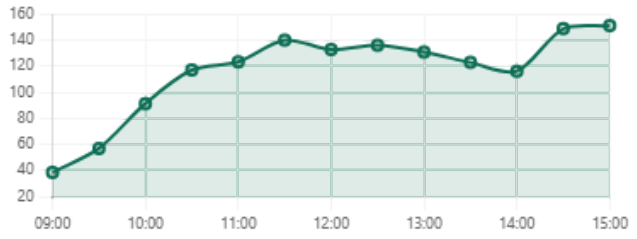
DB TIME (S/S)



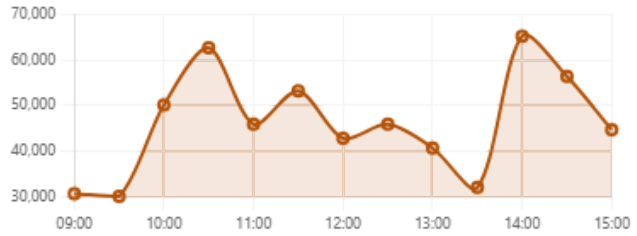
DB CPU (S/S)



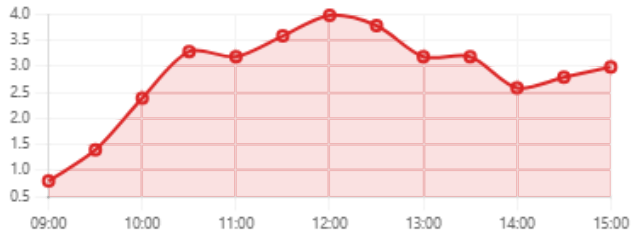
TRANSACTIONS / S



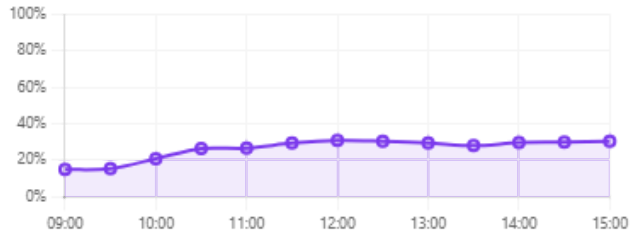
PHYSICAL READS / S



HARD PARSES / S

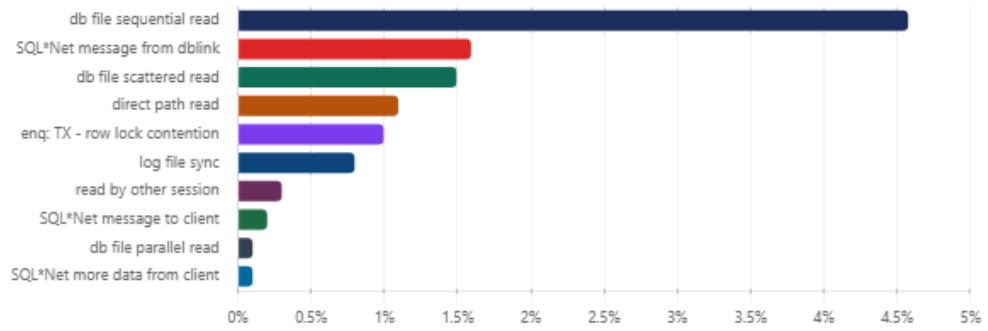


HOST CPU BUSY %



PERFORMANCE ANALYSIS — WAIT EVENTS

TOP WAIT EVENTS — AVG % OF DB TIME



WAIT CLASS DISTRIBUTION — % OF DB TIME

Event	Total Waits	Avg % DB Time	Total Wait (s)
db file sequential read	51,783,439	4.6%	24129.0 s
SQL*Net message from dblink	556,202	1.6%	3659.0 s
db file scattered read	9,219,900	1.5%	7691.0 s
direct path read	6,892,609	1.1%	5080.0 s
enq: TX - row lock contention	3,727	1.0%	5098.0 s
log file sync	2,665,549	0.8%	4148.0 s
read by other session	2,413,318	0.3%	1703.0 s
SQL*Net message to client	172,426,756	0.2%	818.0 s
db file parallel read	1,186,154	0.1%	748.0 s
SQL*Net more data from client	4,020,317	0.1%	556.0 s

Cache Efficiency Metric	Avg Value	Status
Buffer Hit %	99.57	✅
Library Hit %	99.86	✅
Soft Parse %	99.74	✅
Buffer Nowait %	99.99	✅
Execute to Parse %	79.48	⚠️

Source: awr_instance_efficiency

🔍 SQL SEVERITY SUMMARY — TOP SQL

Ranks each SQL by how severe a *single execution* of it is across several dimensions at once (elapsed time, CPU, I/O, parsing) — not by how much it has cost in total. A rarely-run statement (e.g. a once-daily EOD/BOD batch job) can legitimately rank #1 here if that one run is genuinely severe, even though a much more frequent statement costs more in aggregate — that cumulative/volume view is what the Hotspot Areas and Recommendations below are based on instead, so the two can name different SQL IDs without either being wrong — they're answering different questions.

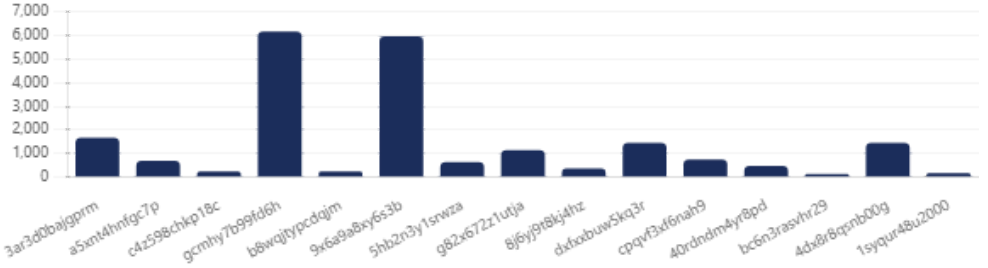
CRITICAL 1 SQL | HIGH 0 SQLs | MEDIUM 2 SQLs | LOW 12 SQLs

Severity	SQL ID	Module	Executions	Total Elapsed (s)	Elapsed/Exec (s)	Total CPU (s)	CPU/Exec (s)	Avg Gets/Exec	Avg Reads/Exec	Severity Score	% of Sev. Score	% of Elapsed Time	Plan	SQL Preview
CRITICAL	3ar3d0bajgprm	PL/SQL Developer	1	1690.3 s	1690.34 s	1648.8 s	1648.83 s	1562487876	0	24276.03	59.1%	%	✓ Stable	BEGIN PROC_EXT_CREDIT_MIS_...
MEDIUM	a5xnt4hnfgc7p	DBMS_SCHEDULER	1	715.9 s	715.93 s	0.0 s	0.0 s	75425373	37784421	5243.99	12.8%	%	✓ Stable	--reportid=160 SELECT T.APPLIC...
MEDIUM	c4z598chkp18c	TataDigitalSFTP.exe	1	259.0 s	259.03 s	217.2 s	217.22 s	74098958	36657263	4957.16	12.1%	%	✓ Stable	Select VW_DIGITAL_DISBURSAL...
LOW	gcmhy7b99fd6h	DBMS_SCHEDULER	7	6196.7 s	885.249 s	2237.5 s	319.64 s	0	16890014	1628.42	4.0%	%	✓ Stable	DECLARE job BINARY_INTEGER :...
LOW	b8wqjtypcdqjm	PL/SQL Developer	1	249.7 s	249.74 s	244.9 s	244.93 s	82723781	0	1364.52	3.3%	%	✓ Stable	BEGIN PROC_WELCOME_CALLI...
LOW	9x6a9a8xy6s3b	DBMS_SCHEDULER	8	6002.9 s	750.361 s	0.0 s	0.0 s	0	7218871	1059.28	2.6%	%	✓ Stable	SELECT VENDOR_CODE, ASSIGN...
LOW	5hb2n3y1srwza	DBMS_SCHEDULER	1	663.9 s	663.94 s	0.0 s	0.0 s	0	13223177	780.51	1.9%	%	✓ Stable	DECLARE job BINARY_INTEGER :...
LOW	g82x672z1utja	DBMS_SCHEDULER	1	1164.5 s	1164.45 s	562.1 s	562.11 s	0	6962475	634.37	1.5%	%	✓ Stable	SELECT TRANSLATE (TO_CHAR(T...
LOW	8j6yj9t8kj4hz	DBMS_SCHEDULER	1	383.1 s	383.1 s	0.0 s	0.0 s	0	1762302	300.31	0.7%	%	✓ Stable	call NEO_CAS_LMS_PRD_TCF.PR...
LOW	dxfxxbuw5kq3r	JDBC Thin Client	8	1471.6 s	183.955 s	975.6 s	121.95 s	3946342	1053804	206.91	0.5%	%	✓ Stable	/* Select distinct new map (la.id ...
LOW	cpqvf3xf6nah9	DBMS_SCHEDULER	1	764.1 s	764.14 s	0.0 s	0.0 s	0	972243	190.66	0.5%	%	✓ Stable	DECLARE job BINARY_INTEGER :...
LOW	40rdndm4yr8pd	DBMS_SCHEDULER	4	499.3 s	124.82 s	436.9 s	109.23 s	9634572	573497	131.89	0.3%	%	✓ Stable	DECLARE job BINARY_INTEGER :...
LOW	bc6n3rasvhr29	OnlineReport_FSL_192.exe	1	150.2 s	150.22 s	118.5 s	118.47 s	0	584421	131.75	0.3%	%	✓ Stable	Select VW_REPORT_192_MIS_ST...
LOW	4dx8r8qsnb00g	SQL Developer	2	1471.3 s	735.64 s	0.0 s	0.0 s	0	0	91.95	0.2%	%	✓ Stable	SELECT DISTINCT C.MOBILE_GR...
LOW	1syqur48u2000	DBMS_SCHEDULER	1	188.2 s	188.23 s	0.0 s	0.0 s	0	0	88.07	0.2%	%	✓ Stable	call NEO_CAS_LMS_PRD_TCF.GS...

Source: awr_sql_summary_mv · Ranked by severity_score — a weighted composite across all SQL statistics sections (elapsed 25% · CPU 25% · buffer gets 15% · disk reads 10% · user I/O wait 10% · unoptimized reads 5% · parse calls 5% · cluster wait 5%), same basis as the AWR SQL Summary Top 10 By Severity dashboard · % of Sev. Score = share of this Top-15 group's combined severity score (drives the row order and Severity badge) · % of Elapsed Time = this SQL's share of the same group's combined total elapsed time — the cumulative-impact lens used in Hotspot Areas/Recommendations instead · Severity: CRITICAL ≥30% | HIGH 15–30% | MEDIUM 5–15% | LOW <5% of severity score

TOP SQL DISTRIBUTION BY SEVERITY

TOP SQL — TOTAL ELAPSED TIME (S)



SEGMENT SEVERITY SUMMARY — TOP OBJECTS BY I/O & CONTENTION SEVERITY

HOT 0 objects | WARM 5 objects | COOL 1 object | NORMAL 3 objects

Severity	Owner	Object Name	Type	Logical Reads	Physical Reads	Direct Reads	Phys Read Req	Table Scans	Unoptimized Reads	Row Lock Waits	Severity Score	% of Total
WARM	NEO_CM_PRD_TCF	GENERIC_EVENT	TABLE	1,025,414,590	2,026,537	874,041	765,876	1	765,876	0	102726368.23	20.4%
WARM	NEO_CM_PRD_TCF	DECISION_REASON_MAPPING	TABLE	899,535,929	0	0	0	0	0	0	89953592.93	17.9%
WARM	NEO_CM_PRD_TCF	APP_PROCESSING_STEP_DATA	TABLE	759,102,496	0	0	0	0	0	0	75910249.6	15.1%
WARM	NEO_CM_PRD_TCF	SYS_C00724502	INDEX	758,301,360	0	0	0	0	0	0	75830136.0	15.1%
WARM	NEO_CAS_LMS_PRD_TCF	ACT_RU_JOB	TABLE	654,328,689	0	0	0	0	0	34	65432870.25	13.0%
COOL	NEO_CM_PRD_TCF	LOAN_TASK_MAPPING_MODEL	TABLE	327,490,907	0	0	0	0	0	0	32749090.76	6.5%
NORMAL	NEO_GCC_PRD_TCF	PORTAL_LOGGING_FIS	TABLE	228,641,904	2,449,090	612,278	0	0	0	0	23109099.95	4.6%
NORMAL	NEO_CM_PRD_TCF	ACT_RU_JOB	TABLE	209,507,224	0	0	0	0	0	0	20950722.47	4.2%
NORMAL	NEO_CM_PRD_TCF	IDX_GENERIC_EVENT_CP	INDEX	158,563,120	0	0	0	0	0	0	15856312.0	3.2%

Source: awr_segment_summary_mv · Ranked by severity_score — a weighted composite of an I/O Group (60%: physical reads 15% · direct I/O 10% · logical reads 10% · CR+current blocks received 10% · table scans 5% · unoptimized reads 5% · read/write requests 3% · optimized reads 2%) and a Contention Group (40%: DB block changes 10% · row lock waits 10% · ITL waits 5% · buffer busy waits 10% · GC buffer busy 5%), same basis as the AWR Segment Summary Top 10 By Severity dashboard · all metric columns are per-snapshot averages · columns with no non-zero values across the displayed rows are omitted · Severity: HOT ≥30% | WARM 10–30% | COOL 5–10% | NORMAL <5% of this Top-15 group's combined severity score

📋 LOAD PROFILE COMPARISON ACROSS SNAPSHOTS

Metric	Unit	09:00	09:30	10:00	10:30	11:00	11:30	12:00	12:30	13:00	13:30	14:00	14:30	15:00	Peak
DB Time (s/s)	s/s	11.2	11.5	16.3	21.3	20.9	23.0	24.4	24.3	23.4	21.4	24.7	25.5	25.4	25.5
DB CPU (s/s)	s/s	10.5	10.8	14.7	18.8	18.8	20.8	21.9	21.5	21.0	19.8	20.8	21.1	21.4	21.9
Transactions/s	/s	38.7	57.2	91.8	117.6	123.9	140.7	133.4	136.8	131.5	123.4	116.8	149.5	151.7	151.7
Executes/s	/s	1331.1	2207.5	4342.8	5404.5	6422.8	8075.4	7860.1	7841.3	7007.8	6755.5	6283.8	7382.9	8049.8	8075.4
Logical reads/s	/s	2170656.1	1466920.9	1950616.8	2524588.3	2540371.4	2765984.3	2895468.5	2820180.6	2777450.9	2712435.3	2683601.0	2762378.4	2809779.9	2895468.5
Physical reads/s	/s	30641.6	30052.9	50212.0	62758.9	45961.8	53309.4	42864.7	45922.8	40693.0	32043.6	65357.5	56470.7	44772.8	65357.5
Hard parses/s	/s	0.8	1.4	2.4	3.3	3.2	3.6	4.0	3.8	3.2	3.2	2.6	2.8	3.0	4.0
Parses/s	/s	140.3	325.6	727.1	1185.4	1369.1	1594.5	1681.5	1940.7	1641.0	1668.0	1509.0	1579.2	1774.9	1940.7

Source: awr_load_profile · Bold red = peak value across snapshots

💿 I/O PROFILE — READS & WRITES BY FUNCTION

I/O Function	Avg Read Req/s	Avg Read MB/s	Avg Write Req/s	Avg Write MB/s	Avg Latency (ms)	Read Share
TOTAL:	4268.5	386.9	427.7	11.4	0.54	
Buffer Cache Reads	2979.1	99.9	0.0	0.0	0.53	
Direct Reads	1222.0	274.9	5.6	0.3	0.64	
Others	65.6	12.1	13.7	3.4	0.6	
LGWR	1.8	0.0	198.2	3.4	0.69	
Direct Writes	0.0	0.0	10.2	1.3	0.36	
Streams AQ	0.0	0.0	0.0	0.0	0.51	
DBWR	0.0	0.0	200.1	3.0	0.0	

Source: awr_iostat_function · Smart Scan Direct Path Buffer Cache Redo DBWR

DIAGNOSTICS — PARSE PRESSURE & REDO SIZING

PARSING PRESSURE INDICATOR

Total parses/s **1318.2**

Hard parses/s **2.9**

Soft Parse Efficiency **99.8%**

Hard parses cause library cache misses and CPU overhead. Target ≥95% soft parse.

REDO LOG SIZING RECOMMENDATION

Avg redo generated **3.2 MB/s**

Recommended log size **3.0 GB**

3.2 MB/s × 1200 s (20-min target) would need a single 3.8 GB log file — capped at a practical 3.0 GB. **At this redo rate, size log files at the cap and add more log GROUPS instead of one oversized file** — expect roughly 3.7 switches/hour at this size; ensure enough groups exist that ARCH/standby apply keeps up before a switch recurs.

ROOT CAUSE ANALYSIS

SYMPTOMS

NEODBPRD/neodbprd_1 degraded — top wait: db file sequential read · DB Time 21.02 s/s · Health GREEN

ROOT CAUSE

Top wait — db file sequential read

EVIDENCE

Snaps: 116883, 116884, 116885, 116886, 116887, 116888, 116889, 116890, 116891, 116892, 116893, 116894, 116895 · Score: 90.7/100 · Top SQL elapsed: 1690.3 s

ACTION PLAN

> Index-Range Scan Pressure: 'db file sequential read' dominates — identify SQL driving high-volume index-range scans on large tables. Check execution plans for NESTED LOOPS on hot tables; consider faster storage for high-traffic datafiles.

Conclusion & Recommendations

CLOUD AI (CLAUDE)

CONCLUSION

AWR analysis of NEODBPRD/neodbprd_1 (19.0.0.0.0, TCLFSLPRDDB1, 76 CPUs, 589.62 GB RAM) across 13 snapshot(s) shows a lightly loaded database. DB Time averaged 21.02 s/s with DB CPU at 18.61 s/s (89.0% CPU ratio). Throughput averaged 116.4 TPS with 2529264.0 logical reads/s and 46236.0 physical reads/s. The dominant wait event is 'db file sequential read'. Host CPU averaged 26.4% busy (I/O wait 1.2%).

AI ANALYSIS SUMMARY — CLOUD AI (CLAUDE) (CLAUDE-HAIKU-4-5)

Performance Summary

NEODBPRD is experiencing significant contention and resource consumption driven by a single dominant SQL statement (9p0fw9b20bv86) that accounts for 59% of elapsed time and 63% of CPU. The database is also suffering from row-lock blocking (1% of DB time, avg 1.4s waits) and two hot tables generating excessive logical and physical I/O. The combination suggests either a problematic query being executed at very high frequency (67k+ times in the snapshot window), inefficient cursor handling forcing re-parses, or both. The overall state is degraded but not yet critical if the dominant SQL can be corrected.

Top 3 Urgent Priorities:

1. **Fix SQL 9p0fw9b20bv86 execution plan** (59% elapsed time, 63% CPU, 67k executions). This single query is the performance bottleneck. Immediate action: obtain the current plan, check for full table scans or inefficient joins, and verify statistics are fresh. If the plan is suboptimal, apply a SQL Profile or Plan Baseline to stabilize it. Expected impact: 30–50% reduction in overall DB load.

2. **Resolve row-lock contention on hot rows** (1% DB time, 1.4s average wait). This is blocking application throughput. Diagnostic priority: run the blocking session query to identify which table/rows are locked and which sessions hold locks. If sequence-backed tables are involved (likely given the root cause note), increase sequence CACHE immediately. Review application transaction scope to eliminate uncommitted transactions holding locks.

3. **Investigate buffer cache fit for hot tables** (GENERIC_EVENT and MESSAGE_EXCHANGE_RECORD account for 23% and 24% of physical reads). These tables are either too large for the buffer pool or are being scanned inefficiently. Confirm whether they should be pinned in a KEEP buffer pool, whether indexes are missing, or whether the access pattern is genuinely sequential. This is lower priority than fixing the SQL but will amplify improvements made to #1.

Root Cause Pattern:

All three priorities are likely driven by the same underlying issue: **SQL 9p0fw9b20bv86 is performing full scans or inefficient range scans on GENERIC_EVENT and/or MESSAGE_EXCHANGE_RECORD.** The high logical I/O (23% of DB activity) drives physical reads, and if the query is holding locks across multiple rows per execution × 67k times, it explains the row-lock contention. The low execute-to-parse ratio (79%) suggests the application is also re-parsing this SQL on every or most calls, adding parser overhead on top of the expensive execution. Address the SQL plan first; improvements cascade to I/O, contention, and parse overhead.

AI-generated narrative summarising the rule-engine findings below — not a replacement for them. Verify against the Hotspot Areas and Recommendations before acting.

HOTSPOT AREAS

> Top wait — db file sequential read

RECOMMENDATIONS

> INDEX-RANGE SCAN PRESSURE

Physical reads — avg 46236/s — buffer cache pressure or full scans

Host CPU — avg 26.4% busy (I/O wait 1.2%)

🔴 Row Lock Contention — Blocked DML Transactions (enq: TX - row lock contention)

🟠 Hot Segment — Excessive Logical I/O (NEO_CM_PRD_TCF.GENERIC_EVENT)

🟠 Hot Segment — High Physical Reads (NEO_CM_PRD_TCF.MESSAGE_EXCHANGE_RECORD)

🟠 High Total Elapsed Time SQL (SQL 9p0fw9b20bv86)

🟠 CPU-Intensive SQL (SQL 9p0fw9b20bv86)

'db file sequential read' dominates — identify SQL driving high-volume index-range scans on large tables. Check execution plans for NESTED LOOPS on hot tables; consider faster storage for high-traffic datafiles.

BUFFER CACHE SIZING

Physical reads at 46236 blocks/s — review buffer cache hit ratio and SGA sizing using the SGA Target Advisory.

COMPARATIVE ANALYSIS

Run a comparison between peak-load and off-peak snapshots to isolate SQL and wait events that deteriorate specifically under load.

ROW LOCK CONTENTION — BLOCKED DML TRANSACTIONS

Sessions are waiting to acquire row-level locks held by other transactions. Causes: long-running uncommitted transactions blocking others; application not committing frequently enough; missing indexes causing lock escalation on lookup tables; sequence cache too low causing hot rows in sequence backing table. Next step: 1. Identify blocking sessions using the diagnostic SQL below — find who holds the lock and who is waiting.

HOT SEGMENT — EXCESSIVE LOGICAL I/O

This segment is being accessed by a very high number of logical reads, indicating it is a frequently-accessed hot object. It is likely the target of many nested loops, index range scans, or full scans. If it is an index, consider whether it is highly selective. If it is a table, investigate whether the full scan is expected. Next step: 1. Identify the top SQLs accessing this segment (cross-reference with Top SQL by Physical Reads).

HOT SEGMENT — HIGH PHYSICAL READS

This segment is generating a large volume of physical disk reads, indicating it is not fitting in the buffer cache or is being accessed via direct reads. For tables: likely full scans. For indexes: heavy random read pattern typical of OLTP with insufficient buffer cache. Next step: 1. Check buffer cache size relative to this segment's size.

HIGH TOTAL ELAPSED TIME SQL

This SQL is consuming a disproportionate share of total database elapsed time. Can be caused by: poor execution plan (full scans, bad join methods); high execution frequency with moderate per-execution cost; data skew causing optimizer estimate errors; missing or outdated statistics; bind variable peeking causing one-size-fits-all plan. Next step: 1. Pull the full execution plan: SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(:sql_id, NULL, 'ALLSTATS LAST')).

CPU-INTENSIVE SQL

SQL is consuming excessive CPU. Typical causes: large hash joins or sort operations in memory; excessive logical I/O (buffer gets) due to missing or inefficient indexes; function calls in WHERE clause preventing index use; complex analytical/window functions on large datasets; PL/SQL function calls within SQL. Next step: 1. Check buffer gets per execution — if > 100K, the SQL is doing excessive logical I/O.

LOW EXECUTE-TO-PARSE RATIO — CURSOR NOT BEING REUSED

Fewer than 90% of executions are reusing an existing cursor — sessions are parsing more than executing. Indicates application is not holding cursors open between calls, possibly closing and reopening cursors with every execution. Next step: 1. Review application connection layer — ensure cursors are held open and reused.

RISK IF UNRESOLVED

Sustained high physical read rates indicate I/O subsystem pressure — storage saturation could cause unpredictable latency spikes under peak load.